RESULTS REPORT · OCTOBER 2026

The Overflow Shelf

How one affordable AI chip learned to handle documents bigger than its own memory. This report explains, in everyday language, what we built, what it costs, what it gained, and what comes next.

AI Infrastructure Engineering · Plain-Language Edition

1 · The problem: a crowded desk

When an AI reads or writes, it keeps the document it is working on inside the memory of the graphics chip that runs it. That memory is fast — but small and expensive. Everything the model is thinking about has to fit there, all at once.

On our hardware, the chip has 96 GB of memory. The AI model itself — the “brain” — permanently occupies about 80.5 GB of that. What is left over, roughly 15 GB, is the “desk” where the document being read must sit. That desk holds the equivalent of about 600 printed pages of text at one time.

Real work keeps bringing bigger material: entire contracts, whole code repositories, stacks of manuals, full books. When a document is too big for the desk, requests either wait in a long queue or are simply turned away. Buying a bigger chip is not much of an answer — the next size up costs several times more for only twice the desk.

80.5 / 96 GB
of chip memory permanently occupied by the model itself

~600 pages
of text that fit on the remaining desk at one time

2 · The idea: an overflow shelf

Every computer also has a second, much larger memory (the “main memory”, hundreds of gigabytes) that is slower but roomy — and almost completely unused while the AI runs. Our upgrade turns it into an overflow shelf for the desk.

The software, built inside the serving engine, constantly watches which parts of the document the model is actually reading. Pages that have not been touched recently are quietly copied to the shelf. The moment the model’s attention turns back, the needed page is already on its way to the desk again — fetched in advance, in batches, so the model never waits. From the model’s point of view, nothing changed; from the desk’s point of view, it suddenly has a bookcase behind it.

> **And it sleeps when it is not needed.** The shelf can stay completely dormant — behaving exactly like standard serving, at full speed — until the day a document arrives that would not fit. Only then does the librarian start working. In production this means one setting, zero downside on ordinary traffic.

3 · Did the answers stay the same?

Yes — we checked this two independent ways before trusting any speed number.

First, every answer produced during the benchmark comparisons was scored for confidence at each word. The scores from shelf-enabled serving were statistically indistinguishable from standard serving: the differences were smaller than the differences between two identical runs of standard serving itself (our measured noise floor). Second, a built-in integrity checker continuously sampled pages moving between desk and shelf and compared them, byte for byte. Across a four-hour stress test: zero mismatches.

4 · The results, in plain numbers

4.1 · On everyday work, standard serving is a touch faster

We ran a fair, side-by-side contest: the same chip, the same documents (each about a 300-page book's worth of text), the same answers. When all the documents comfortably fit on the desk, standard serving wins by about 8–10% — moving pages to and from the shelf is real work, and here it buys nothing. This is exactly why the shelf sleeps by default.

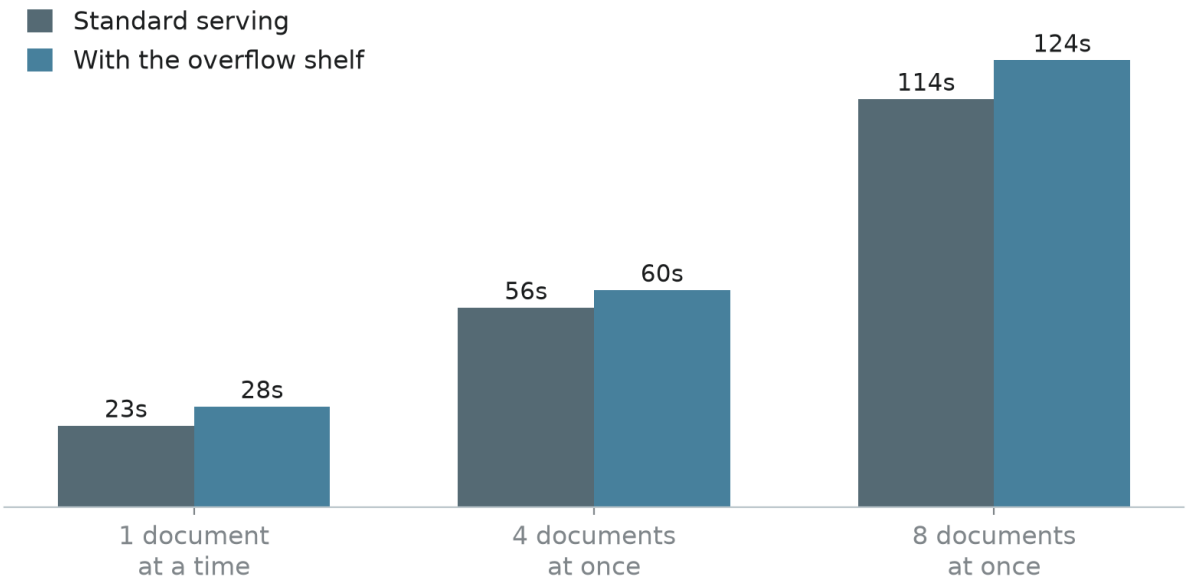


Figure 1 — Time to finish the same batch of large documents. Lower is better. When the desk is roomy enough, the shelf costs 8–10% and is better left asleep.

4.2 · At the edge of the desk, the shelf wins

The interesting jobs are the ones that push the desk right to its limit — documents around 570 pages, arriving several at a time. Standard serving must mostly work on them one by one, because only one fits on the desk. Our system keeps just the actively-read pages of each on the chip and parks the rest, so it can juggle the giants together and finish the whole batch sooner: 39 seconds against 42 for two near-limit documents at once.

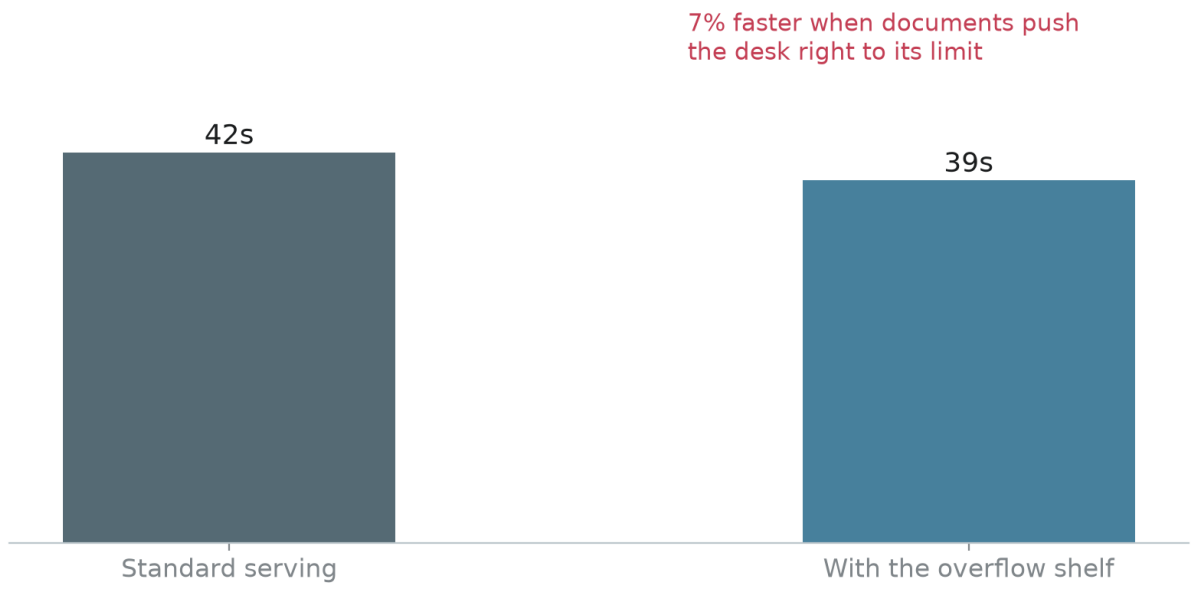


Figure 2 — Two near-limit documents (about 570 pages each) served at the same time. The overflow shelf finishes 7% faster.

4.3 · Most of the book lives on the shelf

While generating answers to a book-sized document, only about 17% of it needed to sit on the chip at any moment. The other 83% rested on the shelf — and generation still ran at 141–215 tokens per second (well over a hundred words per second) on a single, affordable chip.

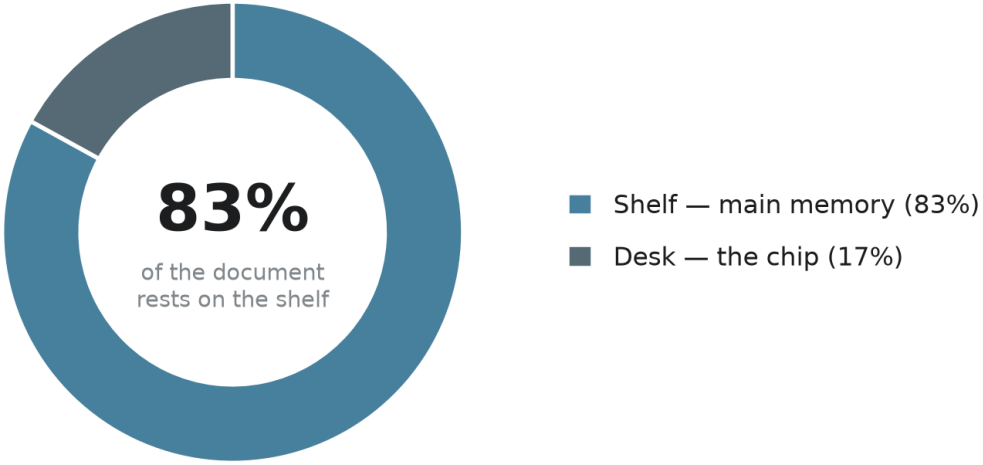


Figure 3 — Where a book-sized document actually sits while the model works on it.

4.4 · We made our own system faster, too

An early version of the shelf copied every page twice and made many small trips. Two changes — copying only what actually changed, and batching shelf trips together — cut the worst-case time on the toughest job by 22%.

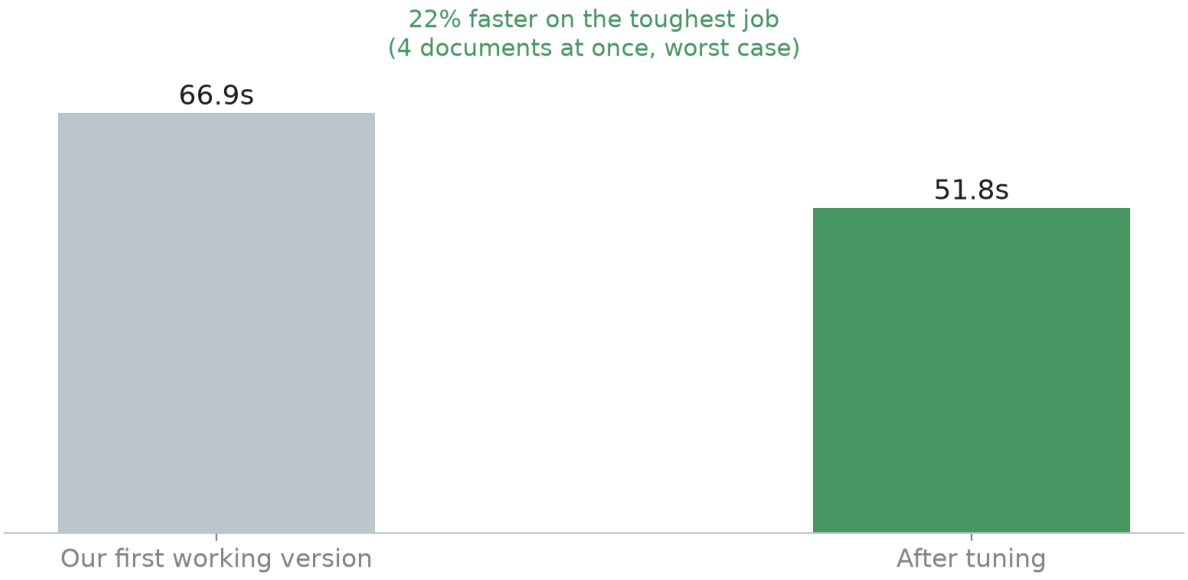


Figure 4 — Our own tuning: worst-case time on the toughest job, before and after.

4.5 · Four hours under continuous load

Finally, we ran the system for four hours straight under a rolling load of book-sized documents, one to eight at a time. The result: 325 documents completed, zero crashes, zero wrong answers, zero stalled requests. The chart below shows the one honest wrinkle — a small, perfectly predictable memory leak that we handle with a two-minute service stop roughly every ten minutes (explained in the next section).

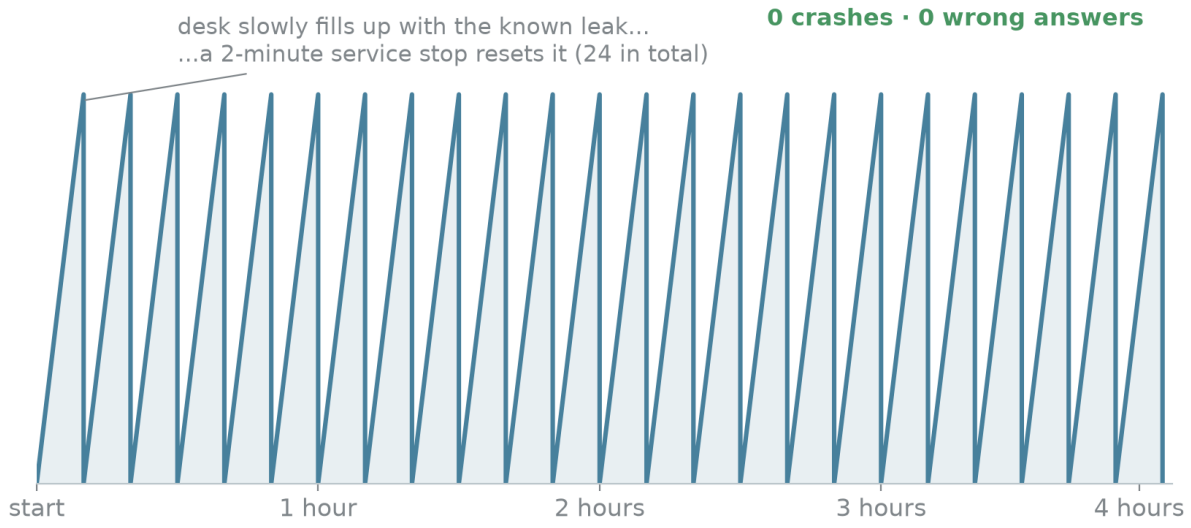


Figure 5 — The four-hour endurance run. Each rising line is desk space being slowly reserved by the known leak; each drop is a two-minute service stop. 24 stops, 325 documents, zero failures.

325	0	24
documents completed in the four-hour endurance run	crashes, wrong answers, or stalled requests	short service stops in four hours — fully automatic

5 · The honest fine print

A small leak remains. Each time a giant document is finished, about 180 tokens' worth of desk space (roughly a page) stays reserved instead of being returned. After about 31 finishes, the server needs a two-minute restart. Today this is fully automated and predictable; we have opened a follow-up engineering ticket to fix it at the root.

For everyday documents, standard is faster. The shelf costs 8–10% when it is not needed — which is why it stays asleep until an oversized document actually arrives.

The very biggest documents need two chips. Material around 780 pages (256k tokens) exceeds what any single chip of this class can take in, in any configuration. Serving it requires two chips cooperating — and a bug in the open-source engine, which we have reported upstream, currently blocks that mode.

Some log warnings are harmless. During heavy moments the engine prints “still working?” health warnings. They are noise; none preceded any failure in testing.

6 · What happens next

Three items, in order: fix the finish-path leak at its root so the service stops disappear (ticket filed today); prepare the two-chip configuration so the largest documents can be served as soon as the upstream bug is fixed; and keep the sleep-until-needed setting as the default in production.

7 · The bottom line

> One affordable chip can now comfortably juggle documents at the very edge of its memory. On work that fits, it runs exactly as before — the shelf sleeps. On the giants, it is 7% faster than standard serving today, with room to grow. And every answer along the way was checked to be identical to the letter.